PROJECT REPORT

Hotel Reservation System

Object-Oriented Programming in C++

Field	Details
Project Title	Hotel Reservation System
Project Code	CPP79
Enrollment Number	IU2641231919
Submitted By	Prapti Bhavsar
Submitted To	Prof. Anjali Chopra
Language / Tools	C++ (C++17), OnlineGDB compiler, UML diagrams

Scanned with OKEN Scanner

1. Project Title

Hotel Reservation System

2. Student Details

Item	Details
Enrollment Number	IU2641231919
Project Code	CPP79
Student Name	Prapti Bhavsar
Submitted To	Prof. Anjali Chopra

3. Problem Statement

Small and medium hotels often manage rooms, guests and bookings with paper registers or loose spreadsheets. This manual approach leads to several real-world problems: the same room can be booked twice, guest details are entered inconsistently or lost, bills are calculated by hand (with mistakes in tax or discounts), and records are difficult to search, correct or remove when a guest changes details or cancels.

The Hotel Reservation System solves this by providing a menu-driven C++ console application that keeps rooms, guests, reservations and bills in one place, validates every input, calculates room charges and taxes automatically, prevents a booked room from being reserved again, and stores all data in files so that nothing is lost when the program is closed.

4. Objectives

- To design and implement a complete hotel reservation application using Object-Oriented Programming in C++.
- To model real-world entities (Room, Guest, Reservation, Bill and Hotel) as classes with proper encapsulation.
- To apply inheritance and runtime polymorphism for different room types (Standard, Suite) and bill types (Regular, Corporate).
- To manage records dynamically using new/delete with automatic array resizing, without fixed limits.
- To provide add, display, search, update and delete operations on records.
- To implement the main transaction of making a reservation, with room-availability checking and automatic bill generation.
- To make data persistent using file handling (save and load).
- To handle invalid input safely using validation and exception handling.
- To document the design with a Use Case Diagram, a Class Diagram and two Sequence Diagrams.

Scanned with OKEN Scanner

5. Features

The program starts by showing a banner with the enrollment number and project code, followed by a main menu. The user selects an operation by entering its number; after each operation the menu is shown again until the user chooses Exit.

No.	Menu Option	Description
1	Add Record	Adds a new Room (Standard or Suite, with room number and price per night) or a new Guest (ID, name, phone, email). Duplicate IDs and invalid values are rejected.
2	Display Records	Shows formatted tables of all rooms (with Available/Booked status), guests, reservations and bills.
3	Search Record	Searches a guest by Guest ID and displays the full details.
4	Update Record	Updates the name, phone and email of an existing guest.
5	Delete Record	Deletes a guest by Guest ID and closes the gap in the array.
6	Main Transaction / Process	Make Reservation: takes Guest ID, Room Number, number of nights and check-in date, checks that the guest and room exist and the room is available, creates the reservation, calculates the room charge and bill total and prints a receipt.
7	Report	Generates a summary report of the hotel records.
8	Save / Load Data	Saves all records to files and loads them back, so data persists between runs.
9	Exit	Closes the program safely and releases all dynamically allocated memory.

Other features

- Input validation (positive IDs, positive prices, non-empty text fields, valid menu choices).
- Exception handling with try/catch around menu operations, displaying a clear ERROR message instead of crashing.
- Different pricing rules: Suite rooms carry a 10% premium; Corporate bills receive a 10% discount with 12% tax, while regular bills use 5% tax.
- Neatly formatted console output using setw, left alignment and fixed two-decimal precision.

6. Classes and Responsibilities

The system is built from eight classes. Room and Bill are base classes; SuiteRoom and CorporateBill are derived classes. Hotel is the controller class that owns and manages all the other objects.

6.1 Room (base class)

Responsibility: Represents a hotel room and knows how to calculate its rental charge.

Data members (protected): roomNumber (int), type (string), pricePerNight (double), available (bool).

Important functions: getRoomNumber(), getType(), getPricePerNight(), isAvailable(), setAvailable(bool); virtual calculateRate(int nights) returns pricePerNight × nights; virtual getRoomCategory() returns "Standard"; virtual serialize() converts the room to a pipe-separated line for file storage.

Scanned with OKEN Scanner

6.2 SuiteRoom (derived from Room)

Responsibility: Represents a premium suite. Inherits everything from Room and overrides the pricing and category.

Important functions: calculateRate(int nights) override returns the standard rate × 1.10 (10% suite premium); getRoomCategory() override returns "Suite".

6.3 Guest

Responsibility: Stores the identity and contact details of a guest.

Data members (private): guestId (int), name, phone, email (string).

Important functions: getters for all fields; setName(), setPhone(), setEmail() used by Update Record; serialize().

6.4 Reservation

Responsibility: Links a guest to a room for a number of nights from a check-in date.

Data members (private): reservationId, guestId, roomNumber, nights (int); checkInDate, status (string, default "Active").

Important functions: getId(), getGuestId(), getRoomNumber(), getNights(), getCheckInDate(), getStatus(), serialize().

6.5 Bill (base class)

Responsibility: Holds the charges for a reservation and calculates the amount payable.

Data members (protected): billId (int), reservationId (int), roomCharge (double), taxRate (double, default 0.05).

Important functions: virtual calculateTotal() returns roomCharge + tax; virtual getBillType() returns "Regular"; getId(), getReservationId(), serialize().

6.6 CorporateBill (derived from Bill)

Responsibility: Bill for corporate customers with a discount and a higher tax rate.

Important functions: constructor sets tax to 12%; calculateTotal() override applies a 10% discount to the room charge and then adds tax; getBillType() returns "Corporate".

6.7 Hotel (controller class)

Responsibility: Owns all rooms, guests, reservations and bills using dynamic arrays and provides every menu operation.

Data members (private): hotelName (string); rooms (Room**), guests (Guest*), reservations (Reservation*), bills (Bill*); a count and a capacity for each array (roomCount, roomCapacity, guestCount, guestCapacity, reservationCount, reservationCapacity, billCount, billCapacity).

Important functions: addRoom(), addGuest(), addReservation(), addBill(); addRecord(), displayRecords(), searchRecord(), updateRecord(), deleteRecord(), makeReservation(), generateReport(), saveData(), loadData(); private resizeRooms(), resizeGuests(), resizeReservations(), resizeBills() that double an array when it is full.

Scanned with OKEN Scanner

6.8 Main program

main() only creates the Hotel object, shows the menu, reads the user's choice, calls the corresponding Hotel function inside a try/catch block, and repeats until Exit is chosen.

7. OOP Concepts Used

7.1 Classes and Objects

Each real-world entity is a class, and objects are created from it at run time. For example, a guest is created as an object and stored in the hotel:

```
addGuest(Guest(id, name, phone, email));
rooms[roomCount] = new SuiteRoom(number, price);   // object created on the heap
```

7.2 Encapsulation

Data members are private or protected and can only be reached through public getters and setters. For instance, the fields of Guest are private, and the only way to change a guest's phone number is setPhone(). Reservation has getters only, so a stored reservation cannot be altered accidentally.

```
class Guest {
private:
    int guestId; string name; string phone; string email;
public:
    string getName() const { return name; }
    void setName(string value) { name = value; }
};
```

7.3 Constructors and Destructor

Every class has a default constructor (safe initial values) and a parameterized constructor. Hotel also uses constructor delegation: Hotel(string name) : Hotel() calls the default constructor first. The Hotel destructor releases all dynamic memory, and the base classes Room and Bill have virtual destructors so derived objects are destroyed correctly.

```
Room(int number, string roomType, double price, bool isAvailable = true);
virtual ~Room() {}

~Hotel() {
    for (int i = 0; i < roomCount; i++) delete rooms[i];
    delete[] rooms; delete[] guests;
    delete[] reservations; delete[] bills;
}
```

7.4 Inheritance

SuiteRoom inherits from Room and CorporateBill inherits from Bill. The derived classes reuse the data members and functions of the base class and add or change only what is different.

```
class SuiteRoom : public Room {
public:
    SuiteRoom(int number, double price, bool isAvailable = true)
        : Room(number, "Suite", price, isAvailable) {}
```


Scanned with OKEN Scanner

```
};

class CorporateBill : public Bill {
public:
    CorporateBill(int id, int reservation, double charge)
        : Bill(id, reservation, charge, 0.12) {}
};
```

7.5 Runtime Polymorphism

calculateRate(), getRoomCategory(), calculateTotal() and getBillType() are virtual and overridden in the derived classes. The Hotel stores rooms as Room* pointers, so the correct version is selected at run time depending on whether the object is a Standard room or a Suite. In the tested run, a Suite of ₹8999 for 2 nights was charged ₹19797.80 (8999 × 2 × 1.10), which is the SuiteRoom version of calculateRate().

```
double calculateRate(int nights) const override {
    return Room::calculateRate(nights) * 1.10;
}

// Hotel::displayRecords()
cout << rooms[i]->getRoomCategory();   // prints "Standard" or "Suite"
```

7.6 Dynamic Memory

All collections are allocated with new[] and grow automatically. The initial capacity is 4. When an array is full, a new array of double the size is allocated, the old elements are copied, and the old array is released with delete[]. Rooms are stored as an array of pointers (Room**) so that base and derived objects can be held together.

```
rooms = new Room*[roomCapacity];
guests = new Guest[guestCapacity];

void resizeGuests() {
    int newCapacity = guestCapacity * 2;
    Guest* temp = new Guest[newCapacity];
    for (int i = 0; i < guestCount; i++) temp[i] = guests[i];
    delete[] guests;
    guests = temp;
    guestCapacity = newCapacity;
}
```

7.7 Exception Handling and Validation

Invalid data is rejected by throwing standard exceptions (invalid_argument, runtime_error), which are caught in main() and shown as an error message.

```
if (number <= 0 || price <= 0)
    throw invalid_argument("Invalid room details.");
if (rooms[i]->getRoomNumber() == number)
    throw runtime_error("Room already exists.");
```


Scanned with OKEN Scanner

8. File Handling

8.1 Data stored

Four kinds of records are stored: rooms, guests, reservations and bills. Each record type is kept in its own text file inside the Data folder (for example rooms, guests, reservations and bills files), with one record per line.

8.2 File format

A pipe-delimited (|) text format is used. Each class has a serialize() function that converts the object to one line. Sample lines:

Record	Format	Sample line
Room	roomNumber|type|price|available	101|Suite|8999.00|0
Guest	guestId|name|phone|email	101|Rahul|9876543210|rahul@example.com
Reservation	id|guestId|roomNo|nights|date|status	1|101|101|2|15-10-2026|Active
Bill	billId|reservationId|roomCharge|taxRate	1|1|19797.80|0.05

The pipe symbol was chosen as the separator because names and emails never contain it, so a line can be split reliably. In the room record, the available field is stored as 1 (available) or 0 (booked).

8.3 Saving process

1. The user selects Save / Load Data from the menu.
2. The files are opened using ofstream (the Data folder is created if needed using the filesystem library).
3. Every object in each array is converted with serialize() and written as one line.
4. The files are closed, and a success message is displayed.

8.4 Loading process

1. The files are opened using ifstream; if a file does not exist, the program continues with empty records.
2. Each line is read with getline() and split into fields using a stringstream on the '|' character.
3. Numeric fields are converted from text to int/double, and a new object is created for the line. A room whose type is "Suite" is created as a SuiteRoom, otherwise as a normal Room.
4. The object is added to the matching dynamic array (which resizes automatically when full).

8.5 Persistence testing

To confirm persistence: records were added (a Suite room 101, guest 101 and a reservation), the data was saved, and the program was closed. When the program was started again and the data was loaded, Display Records showed the same rooms, guests, reservations and bills, with the booked room still marked "Booked".

Scanned with OKEN Scanner

9. UML Diagrams

9.1 Use Case Diagram

Two actors, Guest and Admin, interact with the system. The Admin can perform all operations (add, display, search, update, delete, make reservation, generate report, save/load data), while the Guest is mainly involved in adding, viewing, searching records and making a reservation.

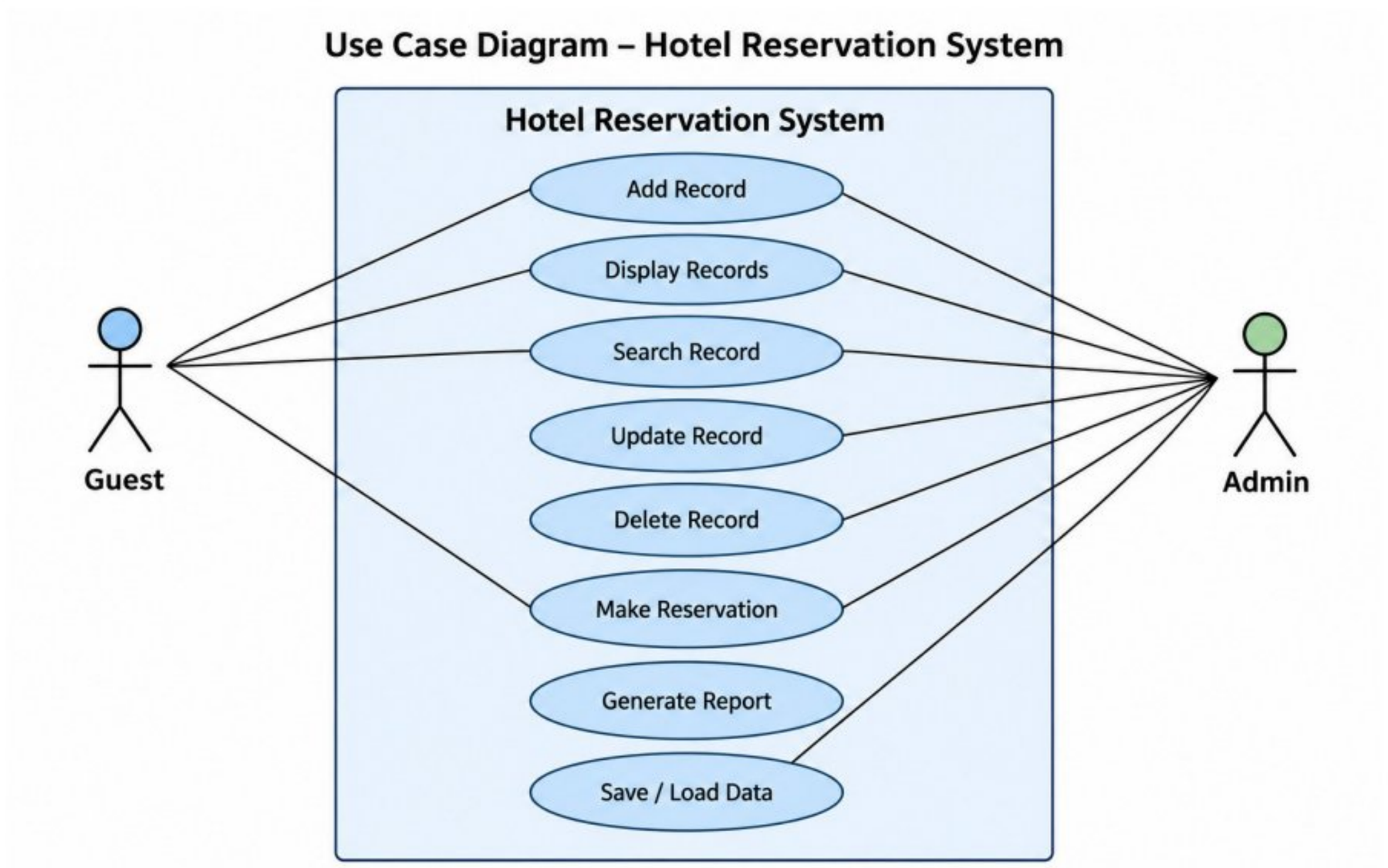


Figure 1: Use Case Diagram – Hotel Reservation System

Scanned with OKEN Scanner

9.2 Class Diagram

The class diagram shows the eight classes with their attributes and methods, the inheritance relationships (SuiteRoom → Room, CorporateBill → Bill) and the associations: one Hotel manages many rooms, guests, reservations and bills, a guest makes zero or more reservations, and a reservation generates a bill.

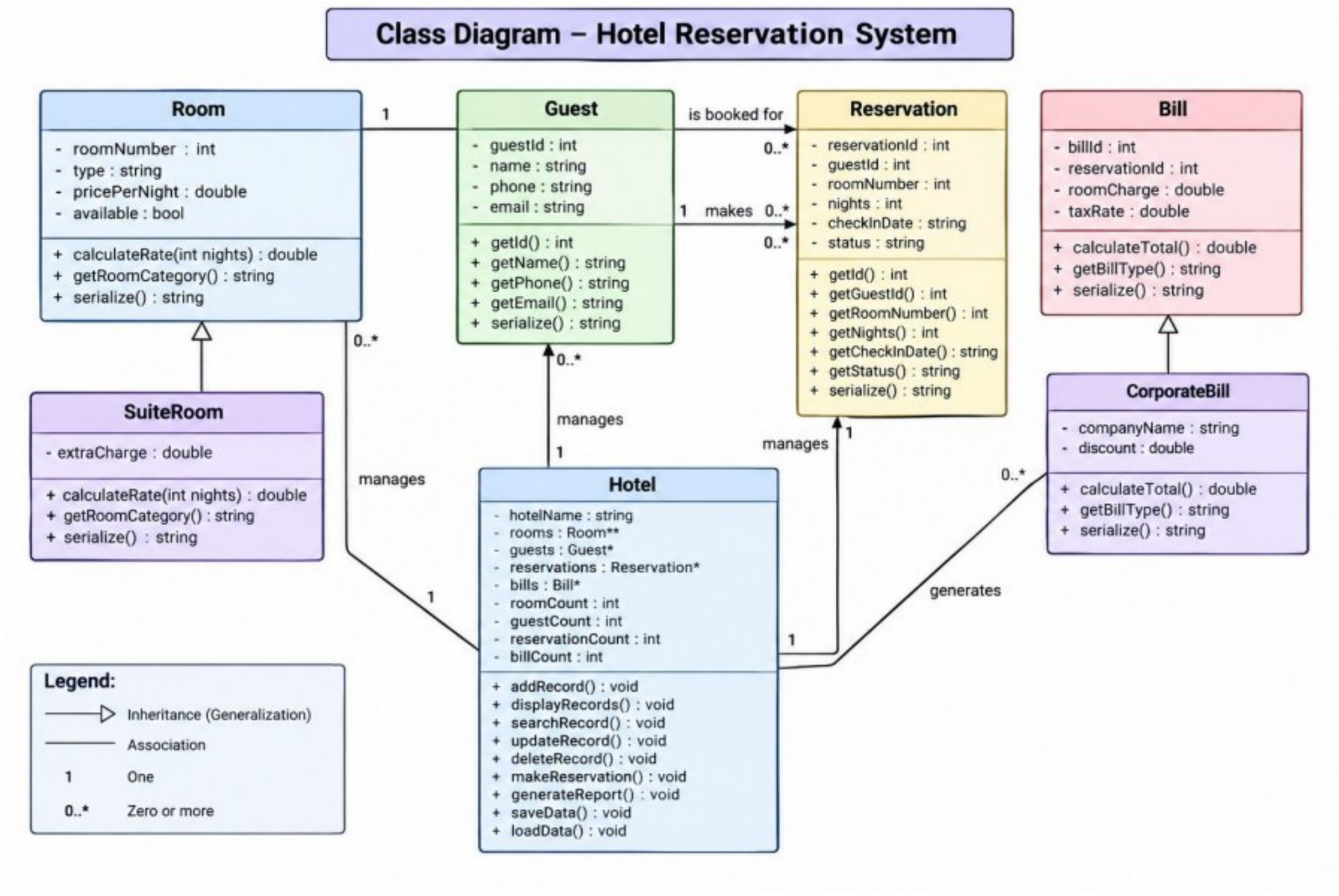


Figure 2: Class Diagram – Hotel Reservation System

Scanned with OKEN Scanner

9.3 Sequence Diagram 1 – Make Reservation (Main Transaction)

The guest selects the Make Reservation option. The menu calls makeReservation() on the Hotel, which checks that the guest exists (checkGuest), checks that the room is available (checkRoom), creates the reservation and returns success, after which "Reservation Completed" is shown to the user.

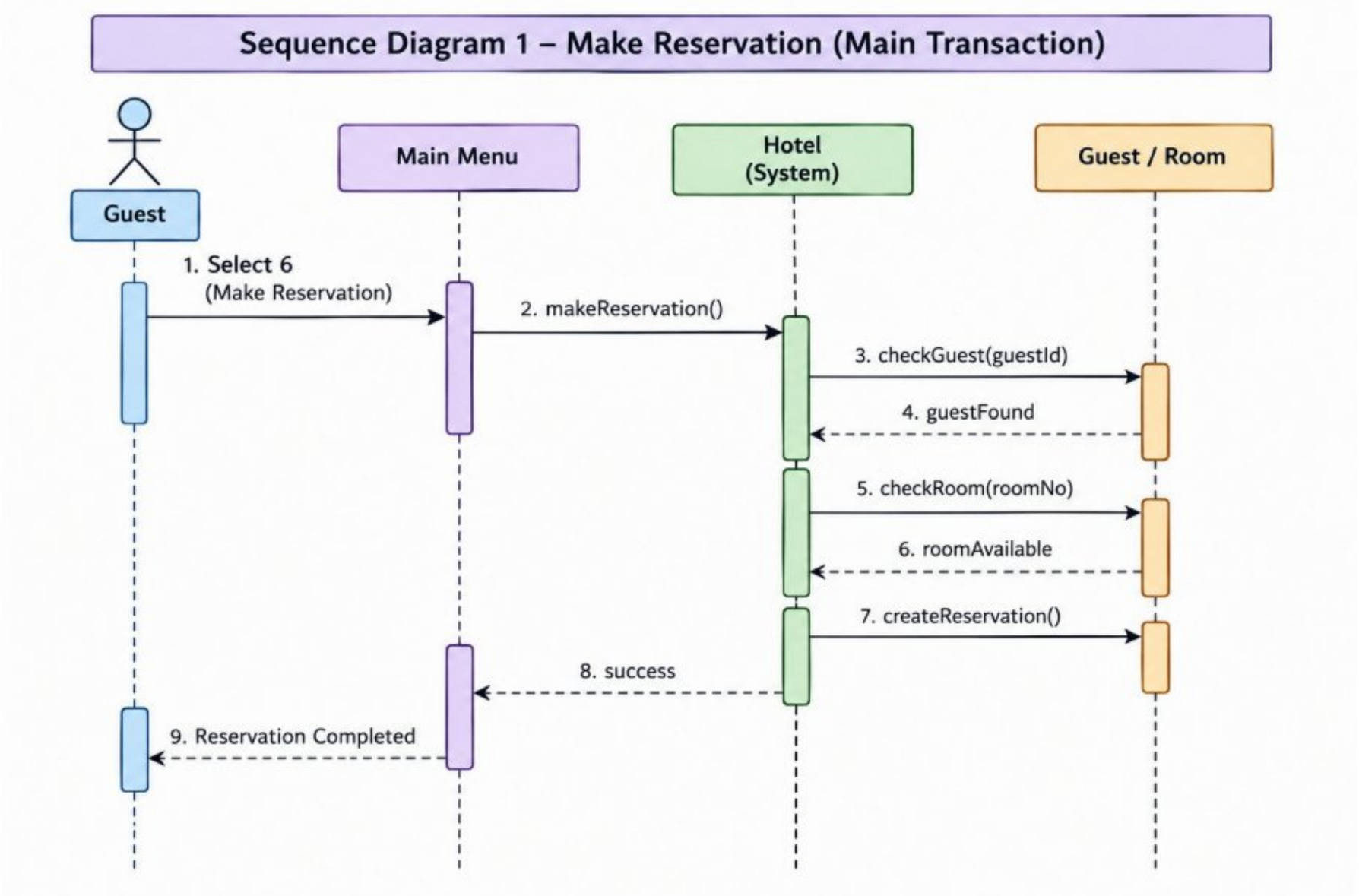


Figure 3: Sequence Diagram 1 – Make Reservation

Scanned with OKEN Scanner

9.4 Sequence Diagram 2 – Search Record

The user selects Search Record. The menu calls searchRecord() on the Hotel, which searches the stored records by ID, receives the result and displays it back to the user through the menu.

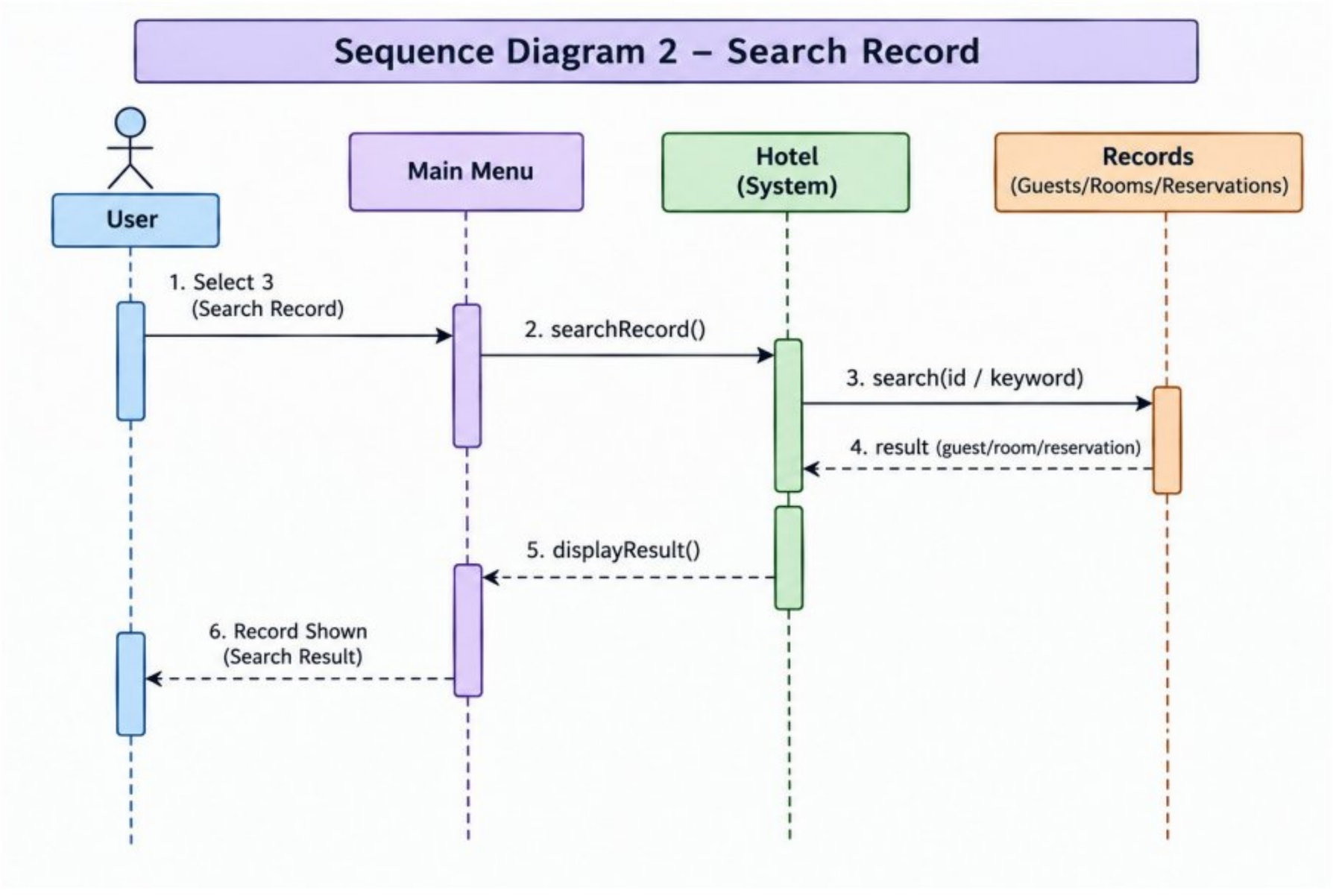


Figure 4: Sequence Diagram 2 – Search Record

10. Program Workflow

1. The program starts and main() creates the Hotel object. Memory for rooms, guests, reservations and bills is allocated dynamically with an initial capacity of 4 each.
2. The banner "HOTEL RESERVATION SYSTEM" with the enrollment number and project code is displayed, followed by the main menu.
3. The user enters a menu choice. The choice is validated; an invalid choice shows an error message and the menu is displayed again.
4. For Add Record, the user chooses Room or Guest, enters the details, the data is validated, and the object is stored in its dynamic array (the array doubles in size if full).
5. For Display, Search, Update and Delete, the Hotel scans its arrays and shows, modifies or removes the matching record.
6. For Main Transaction (Make Reservation), the user enters Guest ID, Room Number, number of nights and check-in date. The system verifies the guest and room, makes sure the room is available, creates the Reservation, marks the room as booked, calculates the room charge using calculateRate() and the total using calculateTotal(), stores the Bill and prints the receipt.
7. For Report, a summary of the hotel data is generated.
8. For Save / Load, the records are written to or read from the data files.
9. Any exception thrown during an operation is caught in main(), an error message is shown, and the program returns to the menu.

Scanned with OKEN Scanner

10. On Exit, the loop ends, the Hotel destructor deletes all dynamically allocated memory, and the program terminates.

11. Screenshots

11.1 Add Record – adding a Suite room and the main menu

A Suite room (number 101, price ₹8999 per night) is added successfully, and the main menu with the enrollment number and project code is displayed again.

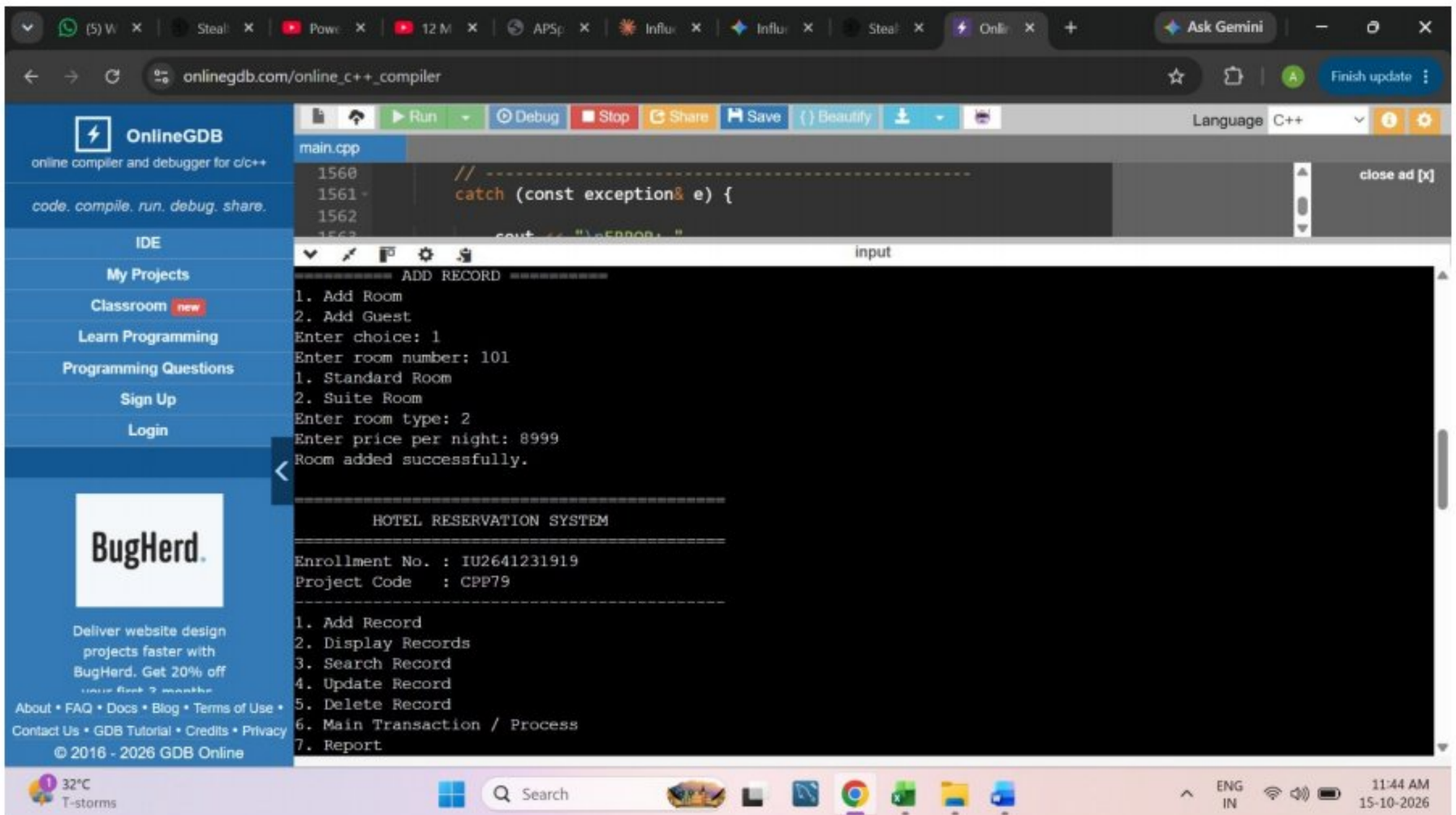


Figure 5: Add Record – Add Room (Suite) and Main Menu

Scanned with OKEN Scanner

11.2 Main Transaction – Make Reservation

Guest 101 reserves room 101 for 2 nights from 15-10-2026. The receipt shows the guest name, reservation ID, room type, room charge (₹19797.80 including the 10% suite premium) and the bill total (₹20787.69 including 5% tax).

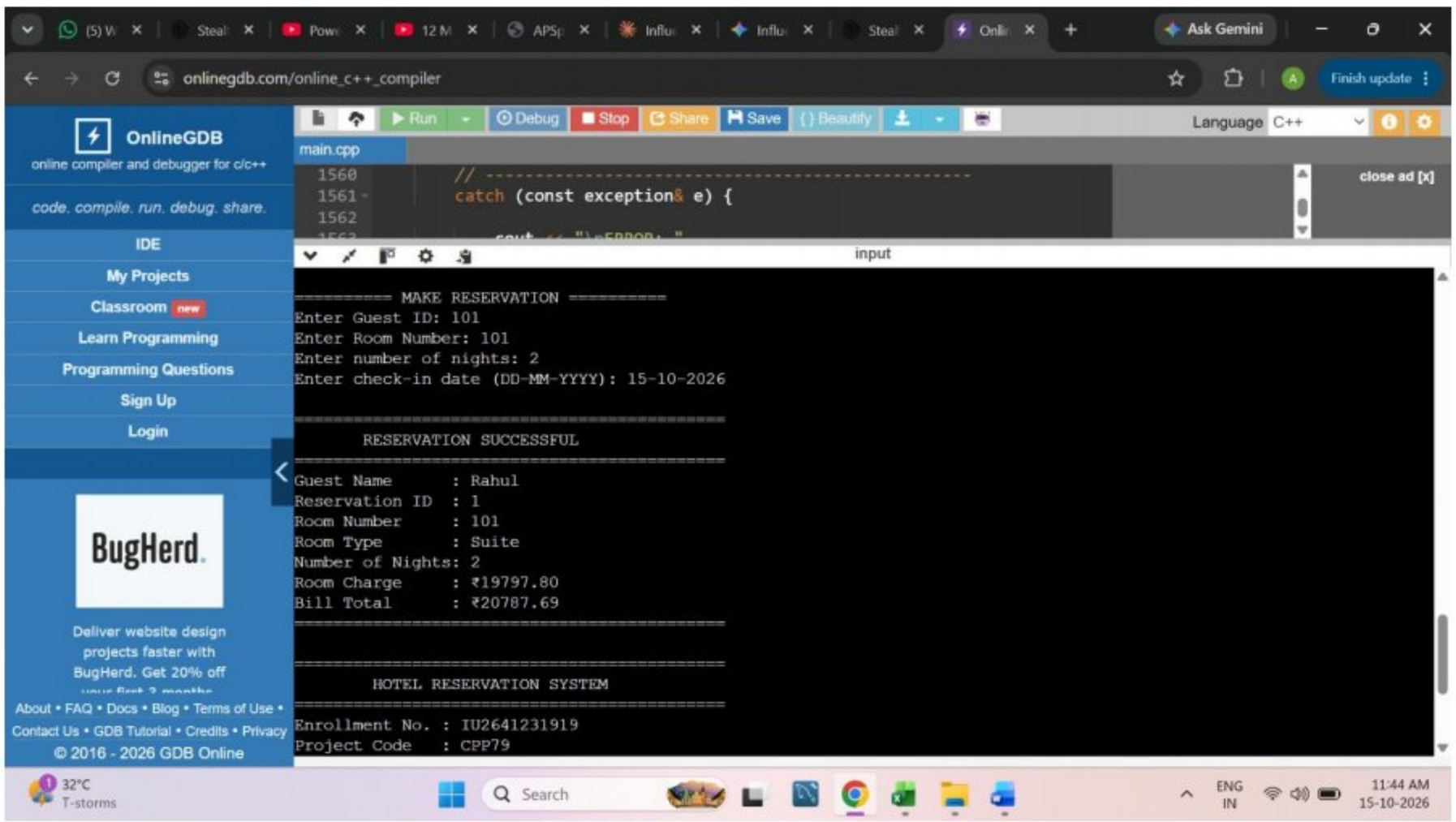


Figure 6: Make Reservation – Successful Reservation and Bill

12. Testing

No.	Test Case	Input	Expected Output	Actual Output
1	Add a Suite room	Choice 1, room 101, type 2, price 8999	Room added successfully.	Room added successfully.
2	Add a duplicate room	Room number 101 again	Error: Room already exists.	Error shown, room not added
3	Add room with invalid price	Room 102, price 0	Error: Invalid room details.	Error shown, room not added
4	Add a guest	ID 101, name Rahul, phone, email	Guest added successfully.	Guest added successfully.
5	Add guest with existing ID	ID 101 again	Error: Guest ID already exists.	Error shown, guest not added
6	Add guest with empty field	Blank email	Error: Guest details cannot be empty.	Error shown, guest not added
7	Search existing guest	Guest ID 101	Guest Found with full details	Guest details displayed
8	Search missing guest	Guest ID 999	Guest not found.	Guest not found.
9	Update guest	ID 101, new name/phone/email	Guest updated successfully.	Guest updated successfully.
10	Delete guest	Guest ID of an existing guest	Guest deleted successfully.	Guest deleted successfully.

Scanned with OKEN Scanner

No.	Test Case	Input	Expected Output	Actual Output
11	Make reservation	Guest 101, room 101, 2 nights, 15-10-2026	Reservation successful; charge = 8999 × 2 × 1.10 = ₹19797.80; total with 5% tax = ₹20787.69	Reservation ID 1, Suite, ₹19797.80 charge, ₹20787.69 total
12	Invalid menu choice	Choice 15	Invalid choice message; menu shown again	Menu displayed again
13	Save, close, reopen, load	Save data, restart program, load data	All records restored	All records displayed after loading

13. Challenges and Solutions

Challenge	Solution
Fixed-size arrays ran out of space when many records were added.	Used dynamic arrays with an initial capacity of 4 and private resize functions that double the capacity, copy the old data and delete the old array.
Storing Standard and Suite rooms together in one list.	Stored rooms as Room* in a Room** array and used virtual functions and a virtual destructor, so the correct class behaviour is used and objects are deleted properly.
Using cin >> and getline() together skipped the name input because a newline stayed in the buffer.	Added cin.ignore(numeric_limits<streamsize>::max(), '\n') before every getline() call.
Invalid or duplicate data (negative price, repeated IDs, empty fields) could corrupt the records.	Added validation checks and threw invalid_argument / runtime_error exceptions, caught in main() so the program shows an error and continues.
Preventing a room from being booked twice.	Each room has an available flag. The reservation checks it before booking and updates it afterwards.
Deleting a record from the middle of an array.	Shifted the following elements one position to the left and decreased the count.
Saving objects of different types to files and rebuilding them on load.	Gave each class a serialize() function, used a pipe-separated format, and recreated the correct class on load (for example, type "Suite" becomes a SuiteRoom).
Clean, aligned console output.	Used setw(), left alignment and fixed with setprecision(2) for tables and currency values.

14. Conclusion

The Hotel Reservation System successfully brings room management, guest management, reservations and billing into a single console application. The project achieved all its objectives: records can be added, displayed, searched, updated and deleted; reservations are validated and billed automatically with correct suite premiums and taxes; invalid input is handled with exceptions; and data is kept in files so it persists between runs.

While building it, I gained practical experience with the main OOP concepts of C++: designing classes with encapsulation, using constructors and destructors, extending classes through inheritance,

Scanned with OKEN Scanner

achieving runtime polymorphism with virtual functions, and managing memory manually with new[] and delete[]. I also learned how to read and write text files, model a system with UML diagrams, and test a program methodically.

In future, the system could be extended with room check-out and cancellation, date-based availability checking, additional room categories, a login system for admin and guest, and a graphical or database-backed interface.

Scanned with OKEN Scanner